Unit 2

Introduction to computer problem solving: Introduction, the problem-solving aspect, top down design, implementation of algorithms, the efficiency of algorithms, the analysis of algorithms.

Fundamental algorithms: Exchanging the values of two variables, counting, summation of a set of numbers, factorial computation, sine function computation, generation of the Fibonacci sequence, reversing the digits of an integer.

Introduction:

Computer problem solving can be summed up in one word it is **demanding**, in this process required much careful planning, logical precision, persistence, and attention to detail. At the same time it can be a challenging, exciting, and satisfying experience with considerable room for personal creativity and expression.

This set of instructions is called program, a program may also be thought of as an algorithm expressed in a programming language. An algorithm is independent of any programming language. To obtain the computer solution to a problem, have to supply the input or data. The program takes the input and manipulates it according to its instructions and eventually produces an output which represents the computer solution to the problem.

An algorithm consists of a set of explicit and unambiguous finite steps which, when carried out for a given set of initial conditions, produce the corresponding output and terminate in a finite time. After studying even a small sample of computer problems it soon becomes obvious that the conscious deapth of understanding.

The Problem-Solving Aspect:

- ✓ Specify the problem
- ✓ Analysis Phase
- ✓ Designing Phase
- ✓ Coding Phase
- ✓ Testing Phase
- ✓ Maintenance Phase

Specify the problem:

This step involves trying to understand the statement of the given problem.

Analysis Phase:

The analysis phase involves identifying the various inputs that are required and the output that is expected by the problem.

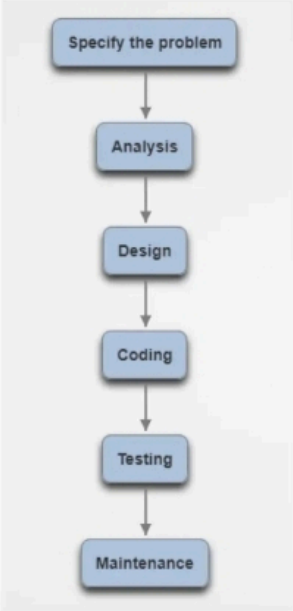
Designing Phase:

The design phase involves a sequence of steps that will allow us to take the given input to get the required output. A flow chart or some other tool can be used to pictorially represent the design step by step.

Coding Phase: The coding phase involves translating the steps identified in the design phase into a machine readable code (program) which is executed to produce the required output. A program is written in a programming language (machine readable code).

Testing Phase:
The testing phase involves verifying if the output received in the coding phase is the required output using the given input.

Maintenance Phase:
A software program that is developed and tested made available to customers. Any change required to this program involves adding small updates. This is the maintenance phase.



Top down Design:

The primary goal in computer problem solving is an algorithm which is capable of being implemented as a correct and efficient computer program. Once we have defined the problem to be solved and we have at least an idea on how to solve it, we can design algorithms using different techniques.

Top-down design or **stepwise refinement** is one such technique for designing algorithms. Top-down design is a strategy that we can apply to take the solution of a computer problem from a vague outline to a precisely defined algorithm and program implementation. It allows us to build our solutions to a problem in a stepwise fashion.

1) Breaking a problem into sub problems

the top-down design suggests that we take the general statements that we have about the solution, and break them down into a set of more precisely defined subtasks. The process of repeatedly breaking a task down into subtasks should continue until we eventually end up with subtasks that can be implemented as program statements.

2) Choice of a suitable data structure

All programs operate on data and consequently, the way the data is organized can have a profound

effect on every aspect of the final solution. The things we must be aware of in setting up data structures are: can it be easily searched, updated, can we recover to an earlier state of the computation, etc.,

Implementation of Algorithms

For a properly designed algorithm, the path of execution should flow in a straight line i.e from top to bottom. Programs implemented in this way are easy to understand, debug and modify.

Use of Procedures to emphasize modularity: It is usually helpful to modularize the program along the lines of Top-down design. This practice allows us to implement a set of independent procedures to perform specific and well-defined tasks.

Choice of Variable names: To make the program meaningful and easier to understand, the names of the variables and constants should be appropriate. For example, if we have to make the changes on the day of the week it is better to use a variable name as **day** instead of using single letter **a** or **b**. A clear definition of all variables and constants at the start of each procedure can also be helpful.

Documentation of Programs: We need to associate brief and accurate comments for each program. The program must specify exactly what responses it requires from the user during execution. A good programming practice is to always write programs so that they can be executed and used by other people.

Program Testing: While testing the program, all the boundary conditions should be verified. It is often not possible to write programs that handle all input conditions that may be given to a particular problem. It is always a good practice to write an algorithm that should satisfy a wide range of possible inputs.

Analysis of Algorithms: Good algorithms generally possess the following qualities and capabilities:

- They are simple and easily understandable by others
- They can be modified easily if necessary
- They have correct and clear defined solutions
- They are economical in the use of computer time, storage and peripherals
- They should run independent of computer
- The solution should be pleasing and satisfied by the designer.

Fundamental algorithms

Write a c program to exchange of two numbers without using third variable

Algorithm:

Step 1: Start
Step 2: Read the value of Number1, Number2
Step 3: Number1= Number1 + Number2
Step 4: Number2= Number1 - Number2
Step 5: Number1= Number1 - Number2
Step 6: PRINT the value of Number1 and Number2
Step 7: Stop

Source Code:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a,b;
        clrscr();
        printf("\nEnter two numbers :");
        scanf("%d%d",&a,&b);
        printf("\n Before swapping, values are a=%3d and b=%3d",a,b);
        a=a+b;
        b=a-b;
        a=a-b;
        printf("\n After swapping, values are a=%3d and b=%3d",a,b);
        getch();
}
```

Input:-

Enter two numbers: 7 9

Output:-

Before swapping, values are a=7 and b=9
After swapping, values are a=9 and b=7

Write a c program to exchange of two numbers using third variable

Algorithm: for using a third variable

Step 1: Start
Start 2: READ num1, num2
Start 3: temp = num1
Start 4: num1 = num2
Start 5: num2 = temp
Start 6: PRINT num1, num2
Start 7: Stop

Source Code:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a,b,temp;
        clrscr();
        printf("\nEnter two numbers :");
        scanf("%d%d",&a,&b);
        printf("\n Before swapping, values are a=%3d and b=%3d",a,b);
        temp=a;
        a=b;
        b=temp;
        printf("\n After swapping, values are a=%3d and b=%3d",a,b);
        getch();
}
```

Input:-
Enter two numbers: 7 9
Output:-
Before swapping, values are a=7 and b=9
After swapping, values are a=9 and b=7

Write a c program to count the digits in a given number.

Step 1: Start
Start 2: READ number
Start 3: Initilize the count ←0
Start 4: repert step 5 untill number become 0 other wise goto step 7
Start 5: devide number with 10
Start 6: count←count+1
Start 7: write count
Start 8: Stop

Source Code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
Long long n;
Int count=0;
Printf(enter the integer);
Scanf("%d",&n);
While(n>0)
{
n=n/10;
count++;
}
Printf("number of digitsis:%d",count);
}
```

Input:-
Enter an integer: 755
Output:-
Number of digits is:3

Write a program to find the sum of the digits of a number

Algorithm:

Step 1: Get number by user
Step 2: Get the modulus/remainder of the number
Step 3: sum the remainder of the number
Step 4: Divide the number by 10
Step 5: Repeat the step 2 while number is greater than 0.

Source Code:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int n ,r ,sum=0;
        clrscr();
        printf("\n Enter the number : ");
        scanf("%d",&n);
        while(n!=0)
        {
                r=n%10;
                sum=sum+r;
                n=n/10;
        }
        printf("\n Sum of the digits  is = %4d",sum);
        getch();
}
```

Input:

Enter the number: 256

Output:

Sum of the digits is = 13

Write a program to compute the factorial of a given number

Algorithm:

Step 1: Start
Start 2: Read n
Start 3: Initialize counter variable i and fact to 1
Start 4: if condition is true go to step 5 otherwise go to step 7
Start 5: calculate fact = fact * i
Start 6: increment counter variable i and go to step 4
Start 7: Write fact.
Start 8: Stop

Source Code:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
                int n,i;
                unsigned long long fact=1;
                clrscr();
                printf("\n Enter a number:");
                scanf("%d",&n);
                if(n<0)
                {
```

```
                Printf("factorial for a negative number doesn't exists");
                }
                else
                {
                for(i=1; i<=n; i++)
                {
                  fact*=i;// fact=fact*i;
                   i++;
                }
                printf("Factorial of %d is = %llu",n,fact);
                getch();
}
```

Input

Enter a number:6

Output

Factorial of 6 is = 720

Write a program to compute the sine function computation

Algorithm:

Step1: Start
Step 2: Initialize int i,n
Step 3: Initialize double sum,x,t
Step 4: Read x and n values
Step 5: find the x value using formula.
X=x*π/180 // 3.14159/180
Step 6: Check the condition using for loop
Step 7: Compute the sin function t-> (t*(-1)*x*x)/(2*i*(2*i+1))
Step 8: Calculate the sin function using sum=sum + t
Step 9: Display sum
Step 10: Stop

Source Code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
  int i,n;
float x,sum,t;
clrscr();
printf("enter the value of x:");
scanf("%f",&x);
printf("enter the value of n:");
scanf("%d",&n);
x=x*3.14159/180;
```

```
t=x;
sum=x;
for(i=1;i<=n;i++)
{
t=(t*(-1)*x*x)/(2*i*(2*i+1));
sum=sum+t;
}
printf("the value of sin(%f)=%0.4f",x,sum);
}
```

Input

Enter the value of x:45
Enter the value of n:4

Output

The value of sin (0.785398) =0.7071

Write a program to generate Fibonacci numbers in the given range.

Algorithm:

Step 1: Start
Step 2: Declare and initialize the necessary variables
Step 3: Check the condition, if the condition is true enter into loop
Step 4:Display F value till the condition get false
Step 5: Stop

Source Code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int n,i,f1=0,f2=1,f;
        clrscr();
        printf(" \n Enter how many fibonacci numbers you want to print : ");
        scanf("%d",&n);
        printf("\n Fibonacci series is :\n");
        printf("%d\t%d",f1,f2);
        for(i=3;i<=n;i++)
        {
                f=f1+f2;
                printf("\t%d",f);
                f1=f2;
                f2=f;
        }
        getch();
```

```
}
```
Input:
Enter how many Fibonacci numbers you want to print: 8

Output:-
Fibonacci series is:
0 1 1 2 3 5 8 13

Write a program to reverse the digits of a number.
Algorithm:

Step 1: Start
Step 2: Declare and initialize the necessary variables
Step 3: Enter the number to reverse
Step 4: Check the condition if the condition is true enter into loop
Step 5: Compute the following logic
reverse=reverse*10;
reverse=reverse+n%10;
n=n/10;
Step 6: Display reverse value
Stop 7: Stop

Source Code:

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int n,reverse=0;
        clrscr();
        printf("\n Enter a number :");
        scanf("%d",&n);
        While (n! =0)
        {
        reverse=reverse*10;
        reverse=reverse+n%10;
                n=n/10;
        }
        printf("\n The reverse of given number is:%d", reverse);
}
```
Input:
Enter a number: 325
Output:

The reverse of given number is: 523

****

Compiler or Interpreter will be used to convert HLL to MLL and vice versa.

Compiler: converts HLL to MLL in one step.

Interpreter: Converts HLL to MLL line by line.

Example: C,C++, Java

Algorithm: It is the step-by-step procedure to solve a given problem. It will accept input, processes and gives output. It is simple to represent and **it is independent of programming language.**

Example:

I. Write an algorithm to exchange the contents of two variable.

1. [Start]
2. [Input two numbers]
   Read(x,y)
3. [Process two swap two numbers]
   t <- x
   x <- y
   y <- t
4. [Output – to print the swapped numbers]
   Write (x,y)
5. [Stop]

II. Write an algorithm to find the area of a triangle given base and height.

1. [Start]
2. [Input – Base and height of the triangle]
   Read(b,h)
3. [Process – to compute the area of the triangle]
   a <- 0.5*b*h
4. [Output the area]

Write (a)

5. [Stop]

III. Write an algorithm to find the area of a triangle given its 3 sides.

1. [Start]
2. [Input – 3 sides of the triangle]

   Read(a,b,c)
3. [Process – to compute half perimeter]

   s<- (a+b+c)/2.0
4. [Process to compute the area]

   a <- sqrt(s * (s-a) * (s-b) * (s-c))
5. [Output the area]

   Write (a)
6. [Stop]

IV. Write an algorithm to compute the area of a circle

1. [Start]
2. [Input – radius of the circle]

   Read(r)
3. [Initialize]

   p <- 3.14
4. [Process – to compute the area of the circle]

   a <- (p * r *r)
5. [Output the area]

   Write (a)
6. [Stop]

V. Write an algorithm to check whether the given number is odd or even

1. [Start]
2. [Input a number]

   Read(n)

3. [Compute the reminder]

   r <- n modulo 2

4. [process to check whether the given number is odd or even using reminder value]

   If (r = 0)

   Write ("Given number is Even")

   Else

   Write ("Given number is Odd")

5. [Stop]

VI. Write an algorithm to compute the sum of first 'n' natural numbers

1. [Start]
2. [Input a number]

   Read(n)

3. [Initialize]

   sum < - 0

4. [compute the sum of first 'n' numbers]

   for i <- 1 to n step 1

   sum <- sum + i

   end for

5. [Display the sum]

   Write(sum)

6. [Stop]

VII. Write an algorithm to compute the factorial of the given number

1. [Start]
2. [Input a number]

   Read(n)

3. [Initialize]

   fact < - 1

4. [compute the factorial of 'n']

```
for  i <- 1 to n   step 1
        fact <- fact* i
end for
```

5. [Display the sum]

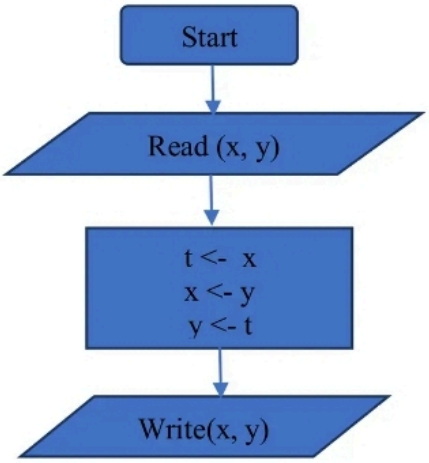
```
Write(fact)
```

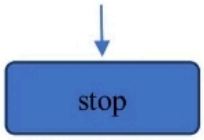
6. [Stop]

Flow chart: It is the pictorial representation of an algorithm. All the steps are drawn in the form of different shapes. Commonly used symbols for flow chart are :

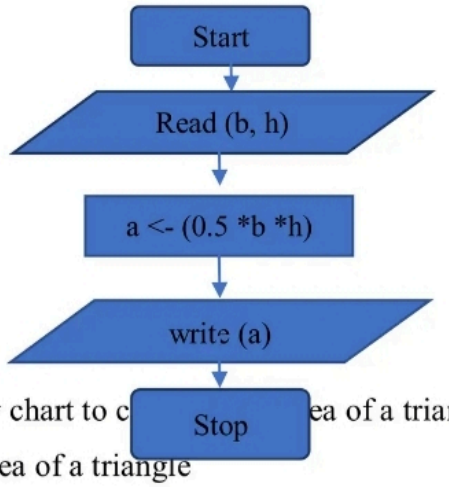
Symbol	Symbol Name	Description
	Flow Lines	Used to connect symbols
	Terminal	Used to start, pause or halt in the program logic
	Input/output	Represents the information entering or leaving the system
	Processing	Represents arithmetic and logical instructions
	Decision	Represents a decision to be made
	Connector	Used to join different flow lines
	Sub function	used to call function

I. Write a flowchart to exchange contents of two variables:

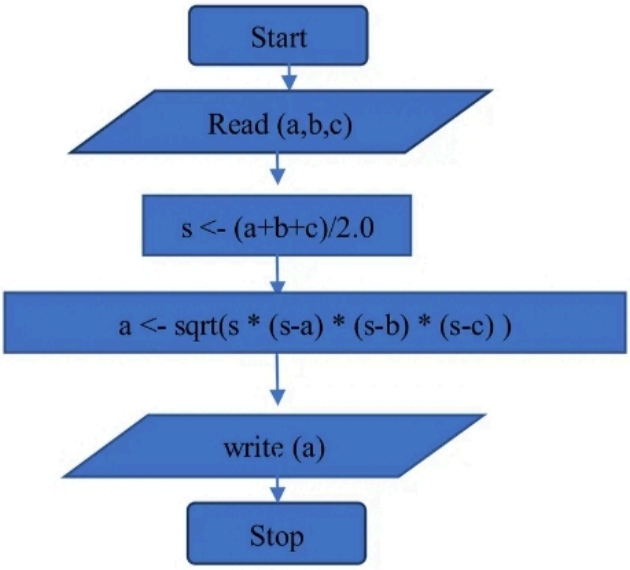




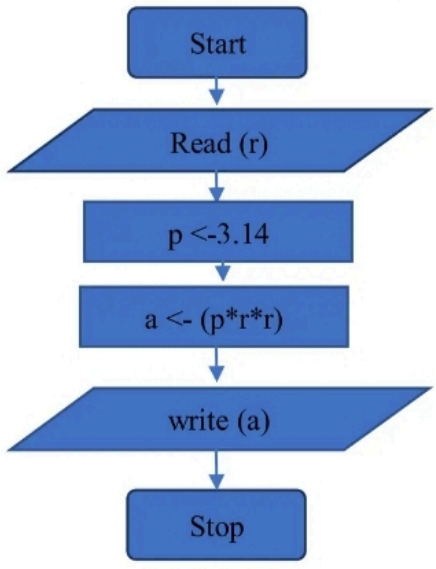
2. Write a flowchart to compute the area of a triangle



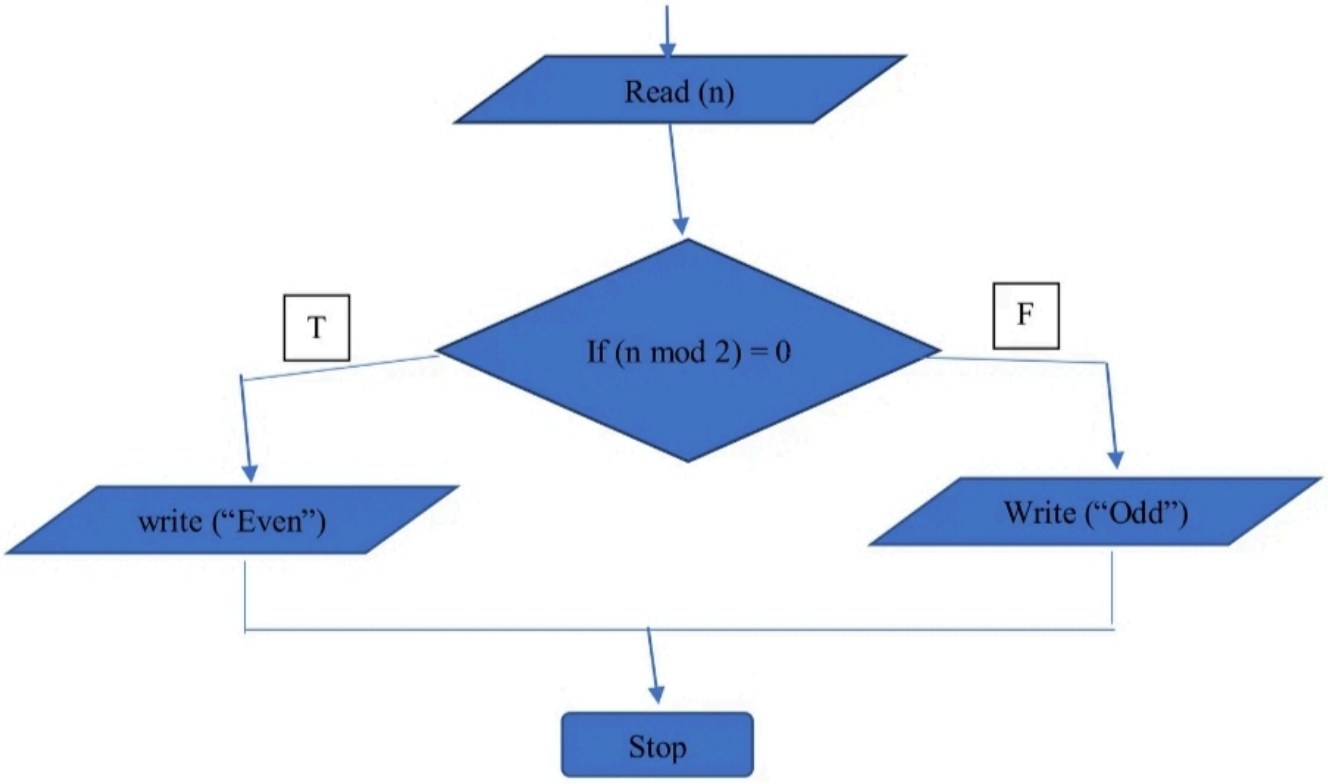
3. Write a flow chart to c ea of a triangle given 3 sides. compute the area of a triangle



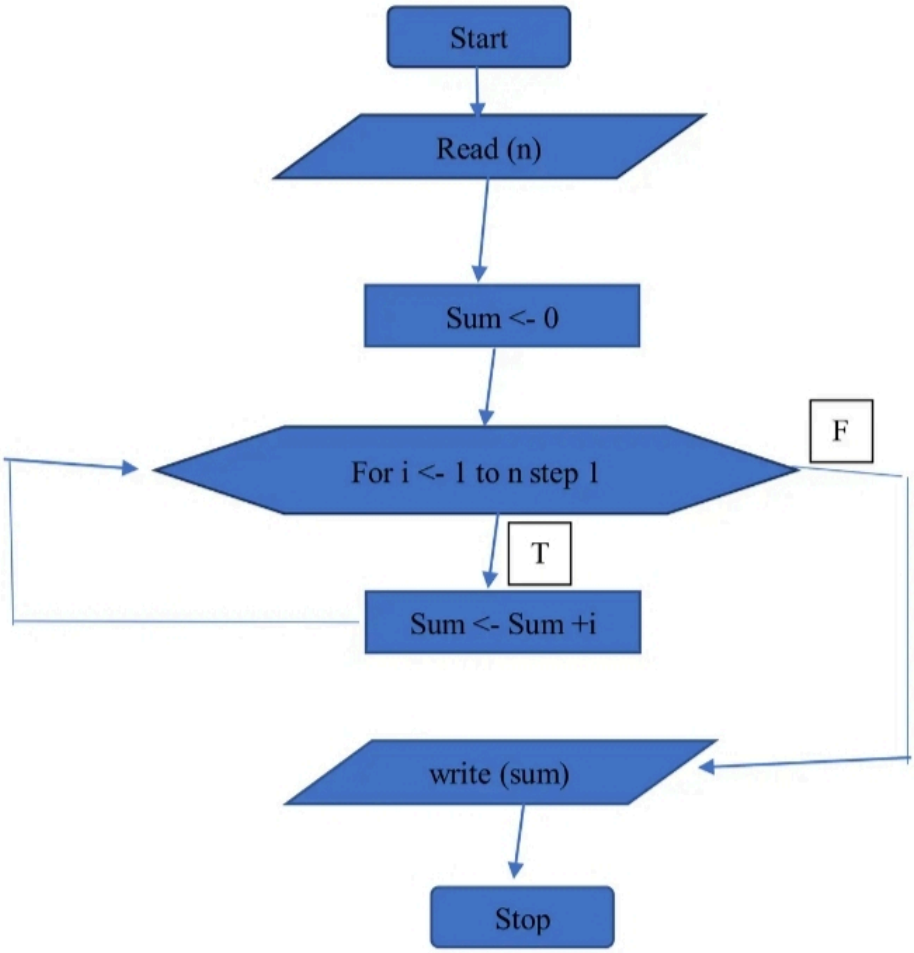
4. Write a flowchart to find the area of a circle



5.Write a flowchart to check whether the given number is odd or even



6.Write the flow chart to compute the sum of first ‘n’ natural numbers



7.Write a flowchart to compute the factorial of a given number

